

Henry V: A linear time algorithm for solving the N-Queens Problem using only 5 patterns

Ali Dehghani

Department of Electrical and Computer
Engineering
University of Hormozgan
Bandar Abbas, Iran
alidehghaniadm@gmail.com

Reza Namvar

Department of CSE/IT
Faculty of Electrical and Computer
Engineering
Shiraz University
Shiraz, Iran
r.namvar@hafez.shirazu.ac.ir

Abdullah Khalili

Department of Electrical and Computer
Engineering
University of Hormozgan
Bandar Abbas, Iran
khalili@hormozgan.ac.ir

Abstract— It has been known that the time complexity of solving the N-Queens problem, a classic problem with many applications in computer science is Nondeterministic Polynomial (NP). Many different approaches have been proposed for solving the problem since its first presentation in 1848 including genetic algorithms, brute force search and propositional logic statements. In this paper, a novel idea called Layouts have been proposed to solve this problem in $\theta(n)$ time complexity. Layouts are patterns for placing queens on the chessboard. Correctness of the proposed approach has been proven for all natural numbers using proof by contradiction and exhaustion. It is shown that by using only 5 layouts, N-Queens with any size can be solved in linear time. The proposed approach has been verified by being applied to 60 different sizes of the N-Queens problem where the size has been chosen by randomly selecting a number between 1000 and 10000.

Keywords— *N-Queens, Time Complexity, Pattern, Proof by Exhaustion, Mathematics Puzzle Introduction*

I. INTRODUCTION

The N-Queens problem, first demonstrated by mathematician Max Bezzel in 1848 [1, 2], is a classic computer science problem. Originally proposed for $N=8$ queens, N-Queens involves arrangement of N queens on an $N \times N$ chessboard so that no two queens threaten one another. (There shouldn't be a row, column, or diagonal with two queens in it). Initially, this problem was only a simple math puzzle. However, today it has many applications in VLSI testing, traffic control, parallel memory storage schemes, deadlock prevention [2, 3], image processing [2, 5], parity-check codes [2, 6], motion estimation [2, 7, 8], modeling of nucleic acids [2, 9] satellite communication [10], neural networks, constraint satisfaction problems [2, 4], and so on. The N-Queen problem, a combinatorial optimization problem as well, has served as a test case for the development of search algorithms.

At first, it would be beneficial to dive deeper into the concept of computational complexity classes. To begin with, Some decision problems are solvable in polynomial time.. That is, given a particular instance, the answer to the problem can be determined in polynomial time. These problems are represented by the complexity class P. The set of all

decision problems with the following property is represented by the complexity class NP: in instances where the answer is "yes", there exists a polynomial-time verifiable proof. It means, by having a specific case of the problem and a witness, we are able to check if the answer is correct in polynomial time. The collection of all NP problems X for which every other NP problem Y can be converted to X in polynomial time is referred to as the NP-Complete class. The final category of problems is known as NP-Hard problems, which are at least as difficult as NP-Complete problems. According to the definition, if a problem Y which is NP-complete can be transformed to another problem X in polynomial time, then problem X is NP-hard. It is crucial to remember that NP-hard problems do not always belong to NP problems class. Any NP-complete problem is clearly NP-hard.

The N-Queens problem is a significant problem with applications in several disciplines being still researched. The exponentially growing time complexity of the problem makes it challenging as well. Owing to challenges in terms of time complexity, backtracking algorithms cannot yield all solutions for large n values. Consequently, the creation of methodologies that can generate one or many answers for high values has become crucial. Despite the fact that the N-Queens problem has already been categorized as either NP-Complete [10] or NP-Hard [11–13], a research has been presented in [14] that aims to solve the problem in linear time on the basis of mathematical constructions, in which no algorithms have been proposed. In this paper, an algorithm is proposed for finding a solution to the N-Queens problem which finds a solution in a linear time. The main motive is using specific patterns (called "Layouts") for placing queens on the chessboard instead of brute-force or other search approaches.

Each Layout works for a specific size of the problem. In the next section, it is shown that by using five layouts proposed in this paper, all instances of the N-Queens problem can be solved in linear time. In section 3, correctness of the proposed approach has been examined using proof by contradiction and exhaustion. Analyzing the time complexity and some simulations are provided in section 4 and lastly, section 5 summarizes the paper.

II. RELATED WORKS

The authors of [15] presented a pattern-based method that can produce at least one distinct solution for all values of n ($n > 3$). This algorithm produces quite fast solutions to the N-Queens problem with $O(n)$ time complexity and can even produce $(n-1)/2$ unique solutions in linear time for some values.

The authors of [16] describe a linear technique used to solve the N-Queens placement problem for an spontaneous arrangement of k queens dispersed consistently on an $N \times N$ checkerboard. An arrangement for back tracking is given which splits the solution matrix into two fundamental levels. Regression formulas are presented to show how basic levels are dependent on the value of n . For the range of n between 7 and 100,000, the number of solutions in which back tracking has not been applied exceeds 35 percent. The paper also presents an algorithm for assessing the correctness of N-Queens problem solutions or arbitrary compositions of k queens.

The Generalized Networks of Evolutionary Polarized Processors was proposed in [17] as a new variation of the Networks of Polarized Evolutionary Processors (NPEP). It may be more efficient and feasible to solve specific difficult problems using GNPEP than NPEP since this model employs its numerical evaluation from a quantitative perspective. To solve the N-Queen problem, the authors implement a GNPEP network. It has been demonstrated by the authors that a given instance of the GNPEP algorithm takes a linear amount of time.

A genetic algorithm that differs from other similar approaches by applying an advanced mutation operation is proposed in [18]. Accidental modifications to the population are caused by the mutation operation. The step is performed after the cross-over operation has been completed. During the mutation operation, parts of the chromosomes that clash with other queens are mutated. It improves fitness by reducing collisions in that chromosome. Like cross-over operations, new altered chromosomes are created and added to the population.

A parallel genetic algorithm for solving the N-Queen problem has been presented in study [19]. The island genetic algorithm and cellular genetic algorithm were parallelized and run.

Research [20] uses heuristic approaches to address the N-Queen problem. Metaheuristic algorithms simulated annealing, genetic algorithm, and tabu search are examined and results are proposed. Additionally, upper bound complexity is calculated. Problem cases with large dimensions can be handled by reducing fitness function time complexity to constant time, resulting in better efficiency.

III. THE LAYOUTS

As stated previously, five layouts are utilized to solve the N-Queens problem for any size. Each layout has its own algorithm and covers different values of N . In the algorithms, variables i and j iterate over columns and rows respectively.

During every iteration, the proper coordinates (x referring to the column and y referring to the row) for placing a new queen is determined based on the characteristics of the layout. The first layout is considered when N is in the form of $6i$, $6i+1$, $6i+4$, or $6i+5$. Layout 1 is shown in Fig.1.a. Moreover, the algorithm of the Layout 1 is presented in Algorithm 1.

Algorithm 1: The algorithm for layout 1

```

k ← 2
for i ← 1 to N do
    if k > N
        k ← 1
    (x,y) ≤ x ← i, y ← k
    k ← k+2

```

The second layout, which is applicable when N is in the form $12i + 8$, has been shown in Fig.1.b. In this case, Algorithm 2 proposes the procedure.

Algorithm 2: The algorithm for layout 2

```

k ← 2
for i ← 2 to [N/2]+1 do
    (x,y) ≤ x ← i, y ← k
    k ← k+2
k ← 3
towQueen ← 0
for i ← [N/2]+2 to N do
    if towQueen == 2
        towQueen ← 0
        k ← k+8
    (x,y) ≤ x ← i, y ← k
    k ← k-2
    towQueen ← towQueen+1
(x,y) ≤ x ← 1, y ← N-3

```

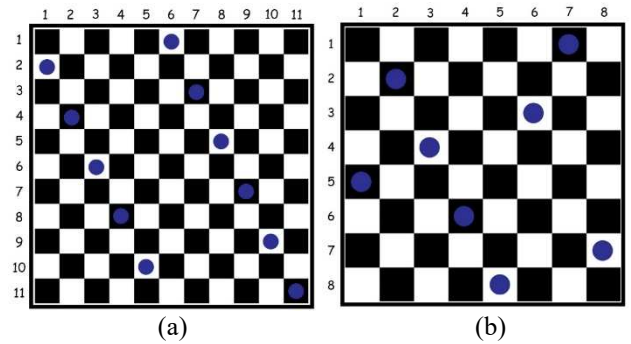


Fig. 1. a) Layout 1, b) Layout 2

When N is in the form of $12i+2$, e.g. 14, 26, 38 and ..., the third layout can solve the problem. For aforementioned sizes the algorithm is shown in Algorithm 3.

Algorithm 3: The algorithm for layout 3

```

k ← 2
for i ← 2 to [N/2]+1 do
    (x,y) ≤ x ← i, y ← k
    k ← k+2
k ← 3

```

```

towQueen ← 0
for i ← [N/2]+2 to N do
    if towQueen == 2
        towQueen ← 0
        k ← k+8
        (x,y) ≤ x ← i, y ← k
        k ← k-2
        towQueen ← towQueen+1
(x,y) ≤ x ← 1, y ← N-1

```

Similarly, Layout 4 can be considered when N is in the form of $12i+9$. The psuedo-code is shown in Algorithm 4.

Algorithm 4: The algorithm for layout 4

```

k ← 1
for i ← 3 to [N/2]+3 do
    (x,y) ≤ x ← i, y ← k
    k ← k+2
k ← 4
towQueen ← 0
for i ← [N/2]+4 to N do
    if towQueen == 2
        towQueen ← 0
        k ← k+8
        (x,y) ≤ x ← i, y ← k
        k ← k-2
        towQueen ← towQueen+1
(x,y) ≤ x ← 1, y ← N-1
(x,y) ≤ x ← 2, y ← N-3

```

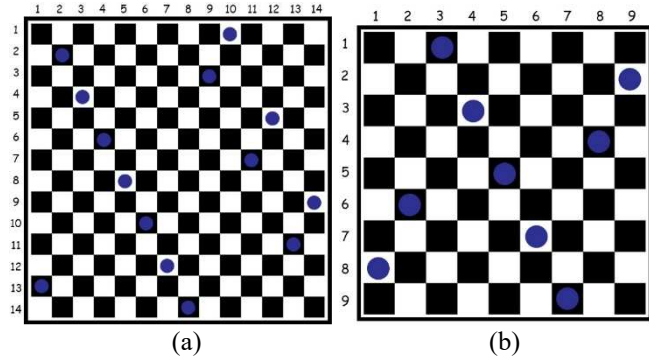


Fig. 2. a) Layout 3, b) Layout 4

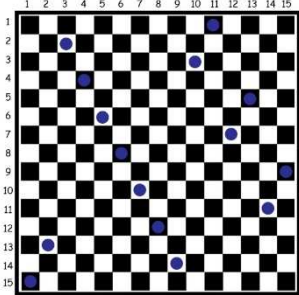


Fig. 3. Layout 5

Finally, the Layout 5 is applicable when N is in the form of $12i+3$, and its procedure is shown in Algorithm 5.

Algorithm 5: The algorithm for layout 5

```

k ← 2
for i ← 3 to [N/2]+2 do
    (x,y) ≤ x ← i, y ← k
    k ← k+2
k ← 3
towQueen ← 0
for i ← [N/2]+3 to N do
    if towQueen == 2
        towQueen ← 0
        k ← k+8
        (x,y) ≤ x ← i, y ← k
        k ← k-2
        towQueen ← towQueen+1
(x,y) ≤ x ← 1, y ← N
(x,y) ≤ x ← 2, y ← N-2

```

IV. CORRECTNESS PROOF

In an effort to prove the correctness of the presented approach, we need to prove the followings:

- Proving that these five layouts can solve any case of the N -Queens problem. For this, we must prove that by putting all the number sets together, all natural numbers are covered.
- Proving that when considering an $N \times N$ chessboard as an $N \times N$ matrix, each layout is producing a correct solution for its associated number set. This means that we must prove each layout works correctly.

Proving (i): based on number theory, we already know that congruence modulo any integer i is an equivalence relation, i.e., it partitions the set of natural numbers $\tilde{N}$ to different congruence classes. A congruence class $[a]_i$ is the set of all integers that have the same remainder as number " a " when divided by " i ". Number sets associated to each layout have been represented by both congruence modulo 6 (Layout 1) and modulo 12 (Layouts 2-5). If it is demonstrated that all the congruence classes are obtained either by modulo 6 or modulo 12, it has been shown that all the natural numbers are covered. Here, only congruence modulo 12 has been used because it is more suitable for showing the coverage.

TABLE I. CONGRUENCE CLASSES MODULO 12 AND CALCULATIONS NEEDED TO PROVE THAT WHOLE NATURAL NUMBERS ARE COVERED

Congruence Class	Calculations	Covered in Layout
$[0]_{12}$	$12i = 6(2i) = 6i'$	1
$[1]_{12}$	$12i + 1 = 6(2i) + 1 = 6i' + 1$	1
$[2]_{12}$	$12i + 2$	3
$[3]_{12}$	$12i + 3$	5
$[4]_{12}$	$12i + 4 = 6(2i) + 4 = 6i' + 4$	1
$[5]_{12}$	$12i + 5 = 6(2i) + 5 = 6i' + 5$	1
$[6]_{12}$	$12i + 6 = 6(2i + 1) = 6i'$	1

Congruence Class	Calculations	Covered in Layout
$[7]_{12}$	$12i + 7$ $= 12i + 6 + 1$ $= 6(2i + 1) + 1$ $= 6i' + 1$	1
$[8]_{12}$	$12i + 8$	2
$[9]_{12}$	$12i + 9$	4
$[10]_{12}$	$12i + 10$ $= 12i + 6 + 4$ $= 6(2i + 1) + 4$ $= 6i' + 4$	1
$[11]_{12}$	$12i + 11$ $= 12i + 6 + 5$ $= 6(2i + 1) + 5$ $= 6i' + 5$	1

According to Table I, all the congruence classes modulo 12 exist in the provided number sets, so all the natural numbers are covered.

Proving (ii): for simplicity, each cell of an $N \times N$ matrix is addressed with an ordered pair (i, j) where i is the number of the row and j is the number of the column of the cell. In addition, we use two other concepts introduced in [14]:

1. Major diagonals of the matrix are those cells with coordinates (i, j) such that $n - j + i = C$, where C is number of the diagonal. Major diagonal number 1 (which is a single cell) is at the top right of the matrix and major diagonal number n is the principal diagonal of the matrix. Clearly there will be a total of $2n - 1$ major diagonals with the last one being the cell at the bottom left of the matrix.
2. Minor diagonals of the matrix are those cells with coordinates (i, j) such that $i + j - 1 = C$, where C is number of the diagonal. Minor diagonal number 1 (which is a single element) is at the top left of matrix. Clearly there will be a total of $2n - 1$ minor diagonals with the last one being the cell at the bottom right of the matrix.

Now we prove that layout 1 produces correct results for its associated number set. Other layouts are proven using a very similar approach to layout 1 and thus, they are not provided here.

Layout 1 consists of two parts: part 1 places queens on the set of cells $(2i, i)$, where $i = 1, 2, \dots, \lfloor \frac{N}{2} \rfloor$ and part 2 places queens on the set of cells $(2i - 1, \lfloor \frac{N}{2} \rfloor + i)$, where $i = 1, 2, \dots, \lfloor \frac{N}{2} \rfloor$. More specifically, part 1 places queens on even-valued rows of the first $\lfloor \frac{N}{2} \rfloor$ columns (i.e. in the left side of the chessboard). Part 2 on the other hand places queens on odd-valued rows of the second $\lfloor \frac{N}{2} \rfloor$ columns (i.e. in the right side of the chessboard). As a result, no two queens are in the same column or row. In case of diagonal threat, the major diagonals used by part 1 are numbered as follows:

$$n - i + 2i = n + i, \quad 1 \leq i \leq \lfloor \frac{n}{2} \rfloor \quad (1)$$

They are unquestionably distinctive. The second part's major diagonals are numbered as follows:

$$n - \lfloor \frac{n}{2} \rfloor - i + 2i - 1 = \lfloor \frac{n}{2} \rfloor + i - 1, \quad 1 \leq i \leq \lfloor \frac{n}{2} \rfloor \quad (2)$$

These are also distinctive.

If a queen from part 1 and a queen from part 2 both use the same major diagonal, the following equation is obtained:

$$n + i = \lfloor \frac{n}{2} \rfloor + i' - 1 \Rightarrow \lfloor \frac{n}{2} \rfloor + 1 = i' - i, \quad 1 \leq i \leq \lfloor \frac{n}{2} \rfloor, \quad 1 \leq i' \leq \lfloor \frac{n}{2} \rfloor \quad (3)$$

Obviously, the highest value of i' can be $\lfloor \frac{n}{2} \rfloor$, while the lowest value of i can be 1. Thus, it is not possible that the difference of i and i' be at least $\lfloor \frac{n}{2} \rfloor + 1$. This means that for part (1) and (2), no two queens are a threat to each other. Thus the idea that a queen from the first part uses the same major diagonal as a queen from the second part should be abandoned.

Part 1 includes the following minor diagonals:

$$2i + i - 1 = 3i - 1, \quad 1 \leq i \leq \lfloor \frac{n}{2} \rfloor \quad (4)$$

which are unique.

Part 2 includes the following minor diagonals:

$$2i - 1 + \lfloor \frac{n}{2} \rfloor + i - 1 = 3i + \lfloor \frac{n}{2} \rfloor - 2, \quad 1 \leq i \leq \lfloor \frac{n}{2} \rfloor \quad (5)$$

These are also distinctive.

If a queen from part 1 and a queen from part 2 both use the same minor diagonal, the following equation is obtained:

$$3i' - 1 = 3i + \lfloor \frac{n}{2} \rfloor - 2 \Rightarrow \lfloor \frac{n}{2} \rfloor - 1 = 3(i' - i), \quad 1 \leq i \leq \lfloor \frac{n}{2} \rfloor \quad (6)$$

For the equation to be satisfiable, variable n should be in the form of either $6k + 2$ or $6k + 3$. None of the needed forms is a part of the number set specified for layout 1. Thus, a queen from part 1 can never use the same minor diagonal as a queen from the second part. So, it can be stated that layout 1 produces correct solutions to the N-Queens problem for its specified number set.

V. TIME COMPLEXITY ANALYSIS AND SIMULATIONS

As it can be easily understood from the pseudo-codes provided in Tables 1-3, all the layouts have a simple "for" clause. Thus, independent of the selected layout, the solution's time complexity is $\theta(n)$. We applied the proposed approach on 60 cases of the N-Queens problems where N was randomly selected from the numbers between 1000 and 10000 and determined the layout that solves them. Correctness of all the cases has been verified by writing an algorithm to check whether placed queens threat each other or not. Table II presents the 60 cases.

TABLE II. LAYOUTS FOR RANDOM NUMBERS BETWEEN 1000 AND 10000

9335-Queen : Layout 1	2390-Queen : Layout 3	62419-Queen : Layout 1
--------------------------	--------------------------	---------------------------

41026-Queen : Layout 1	57753-Queen : Layout 4	24384-Queen : Layout 1
30619-Queen : Layout 1	74066-Queen : Layout 3	57557-Queen : Layout 1
5132-Queen : Layout 2	93580-Queen : Layout 1	96937-Queen : Layout 1
19076-Queen : Layout 2	15577-Queen : Layout 1	18602-Queen : Layout 3
67269-Queen : Layout 4	56802-Queen : Layout 1	2227-Queen : Layout 1
48594-Queen : Layout 1	56524-Queen : Layout 1	60505-Queen : Layout 1
35203-Queen : Layout 1	73425-Queen : Layout 4	54244-Queen : Layout 1
99288-Queen : Layout 1	89174-Queen : Layout 3	18519-Queen : Layout 5
54887-Queen : Layout 1	19950-Queen : Layout 1	86450-Queen : Layout 3
95760-Queen : Layout 1	67955-Queen : Layout 1	71038-Queen : Layout 1
56355-Queen : Layout 5	57947-Queen : Layout 1	74687-Queen : Layout 1
3213-Queen : Layout 4	38259-Queen : Layout 5	53533-Queen : Layout 1
29898-Queen : Layout 1	67227-Queen : Layout 5	73442-Queen : Layout 3
58765-Queen : Layout 1	41180-Queen : Layout 2	6867-Queen : Layout 5
86898-Queen : Layout 1	76355-Queen : Layout 1	7183-Queen : Layout 1
70756-Queen : Layout 1	10764-Queen : Layout 1	65161-Queen : Layout 1
39965-Queen : Layout 1	24601-Queen : Layout 1	52265-Queen : Layout 1
64845-Queen : Layout 4	2253-Queen : Layout 4	71900-Queen : Layout 2
57504-Queen : Layout 1	63417-Queen : Layout 4	33100-Queen : Layout 1

VI. CONCLUSION

Essentially, the N-Queens problem consists of positioning n queens on an $N \times N$ chessboard with no two queens occupying the same row, column, or diagonal at the same time. Consequently, each row, column, and diagonal may only contain a single queen.

In this paper, five linear time patterns called layouts were proposed to solve the N-Queens problem for all sizes. Correctness of the approach was proved by using “proof by exhaustion” and its linear time complexity was shown. Solutions for multiple instances of the N-Queen problem with different values of N have been generated, and the results have confirmed the reliability of our proposed method.

In our future work, we aim to compare the execution time of our proposed method with other approaches.

REFERENCES

- [1] M. Bezzel, "Proposal of 8-queens problem," *Berliner Schachzeitung*, vol. 3, no. 363, 1848.
- [2] J. Bell and B. Stevens, "A survey of known results and research areas for N-Queens," *Discrete Mathematics*, vol. 309, no. 1, pp. 1-31, 2009/01/06/ 2009, doi: <https://doi.org/10.1016/j.disc.2007.12.043>.
- [3] C. Erbas, M. M. Tanik, and Z. Aliyazicioglu, "Linear congruence equations for the solutions of the N-Queens problem," *Information Processing Letters*, vol. 41, no. 6, pp. 301-306, 1992/04/17/ 1992, doi: [https://doi.org/10.1016/0020-0190\(92\)90156-P](https://doi.org/10.1016/0020-0190(92)90156-P).
- [4] C. Erbas, S. Sarkeshik, and M. M. Tanik, "Different perspectives of the N-Queens problem," *Proceedings of the 1992 ACM annual conference on Communications*, pp. 99-108, 1992, doi: <https://doi.org/10.1145/131214.131227>.
- [5] C.-N. Wang, S.-W. Yang, C.-M. Liu, and T. Chiang, "Fast motion estimation using N-Queen pixel decimation," 2006. [Online]. Available: <https://patents.google.com/patent/US7027511>
- [6] P. Li, Z. Guangxi, and L. Xiao, "The low-density parity-check codes based on The N-Queen problem," *Proceedings of the 2004 ACM workshop on Next-generation residential broadband challenges - NRBC '04*, pp. 37-41, 2004, doi: <https://doi.org/10.1145/1026763.1026771>.
- [7] C.-N. Wang, S.-W. Yang, C.-M. Liu, and T. Chiang, "A hierarchical decimation lattice based on N-Queen with an application for motion estimation," *IEEE signal processing letters*, vol. 10, no. 8, pp. 228-231, 2003, doi: <https://doi.org/10.1109/LSP.2003.814403>.
- [8] W. Chung-Neng, Y. Shin-Wei, L. Chi-Min, and C. Tihao, "A hierarchical N-Queen decimation lattice and hardware architecture for motion estimation," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 14, no. 4, pp. 429-440, 2004, doi: <https://doi.org/10.1109/LSP.2003.814403>.
- [9] K. Yamamoto, Y. Kitamura, and H. Yoshikura, "Computation of statistical secondary structure of nucleic acids," *Nucleic Acids Res.*, vol. 12, no. 1 Part1, pp. 335-346, 1984, doi: <https://doi.org/10.1093/nar/12.1Part1.335>.
- [10] R. Jha, D. Das, A. Dash, S. Jayaraman, B. K. Behera, and P. K. Panigrahi, "A Novel Quantum N-Queens Solver Algorithm and its Simulation and Application to Satellite Communication Using IBM Quantum Experience," *arXiv e-prints*, p. arXiv:1806.10221. [Online]. Available: <https://arxiv.org/abs/1806.10221>
- [11] H. Shah-Hosseini, "The intelligent water drops algorithm: A nature inspired swarm-based optimization algorithm," *International Journal of Bio-Inspired Computation*, vol. 1, pp. 71-79, 01/01 2010, doi: <https://doi.org/10.1504/IJBIC.2009.022775>.
- [12] T. Nakaguchi, K. Jin'no, and M. Tanaka, "Hysteresis Neural Networks for N-Queens Problems," *IEICE Transactions on Fundamentals of Electronics Communications and Computer Sciences*, vol. E82-A, pp. 1851-1859, 09/25 1999.
- [13] J. Mańdziuk, "Solving the N-Queens problem with a binary Hopfield-type network," *Biol. Cybern.*, vol. 72, no. 5, pp. 439-445, 1995/04/01 1995, doi: <https://doi.org/10.1007/BF00201419>.
- [14] E. J. Hoffman, J. C. Loessi, and R. C. Moore, "Constructions for the Solution of the m Queens Problem," *Mathematics Magazine*, vol. 42, no. 2, pp. 66-72, 1969/03/01, 1969, doi: <https://doi.org/10.1080/0025570X.1969.11975924>.
- [15] Karabulut, B., Ergüzen, A. & Ünver, H. M. (2022). A Linear Time Pattern Based Algorithm for N-Queens Problem. *Politeknik Dergisi*, 25 (2), 615-622. DOI: 10.2339/politeknik.762967
- [16] Grigoryan, E. (2019). Linear algorithm for solution N-Queens Completion problem. *ArXiv, abs/1912.05935*.
- [17] Arroyo Montoro, F., Gómez-Canaval, S., Jiménez Vega, K., & Ortega de la Puente, A. (2020). A linear time solution for N-Queens problem using generalized networks of evolutionary polarized processors. *International Journal of Foundations of Computer*

- Science*, 31(01), 7–21.
<https://doi.org/10.1142/s0129054120400018>
- [18] Jain, V., Prasad, J. S. (2018). Solving N-Queen problem using genetic algorithm by advance mutation operator. *International Journal of Electrical and Computer Engineering (IJECE)*, 8(6), 4519.
<https://doi.org/10.11591/ijece.v8i6.pp4519-4523>
- [19] Taheri Sarvetamin, M., Khatibi, A., & Zahedi, M. (2018). A New Approach to Solve N-Queen Problem with Parallel Genetic Algorithm. *Journal of Advances in Computer Engineering and Technology*, 4(2), 69-78.
- [20] Martinjak, I., Golub, M. (2007). Comparison of heuristic algorithms for the N-Queen problem. 2007 29th International Conference on Information Technology Interfaces. <https://doi.org/10.1109/iti.2007.4283>